

异常、命名空间

教 师：冀荣华

单 位：人工智能系



主要内容

一、异常

二、命名空间



一、异常

语法错误

运行错误

事先分析程序运行时可能出现的各种意外情况，并给出相应的处理方法

——异常处理

可以预见可能发生在什么地方，但是无法确知怎样发生和何时发生



一、异常

```
#include <iostream.h>
#include "math.h"
double triangle(double a,double b,double c)
{
    double area;
    double s=(a+b+c)/2;
    area=sqrt(s*(s-a)*(s-b)*(s-c));
    return area;
}
void main()
{
    double a,b,c;
    cin>>a>>b>>c;
    while(a>0&&b>0&&c>0)
    {
        cout<<triangle(a,b,c)<<endl;
        cin>>a>>b>>c;
    }
}
```

输入

1 2 1

2 3 2

0 1 2



一、异常

异常处理机制

抛出（**throw**）语句格式：

throw 表达式;

抛出（**throw**）

检查（**try**）-捕捉（**catch**）结构：

检查（**try**）

try

{ 被检查的语句 }

捕捉（**catch**）

catch(异常信息类型 [变量名])

{ 进行异常处理的语句 }



```
#include <iostream.h>
```

```
#include "math.h"
```

```
double triangle(double a,double b,double c)
```

```
{ double s=(a+b+c)/2;
```

```
  if(a+b<=c ||b+c<=a||c+a<=b)
```

```
    throw a;
```

```
  return sqrt(s*(s-a)*(s-b)*(s-c)); }
```

```
void main()
```

```
{ double a,b,c;
```

```
  cin>>a>>b>>c;
```

```
  try{
```

```
    while(a>0&&b>0&&c>0)
```

```
    {      cout<<triangle(a,b,c)<<endl;
```

```
          cin>>a>>b>>c;      }
```

```
  catch(double)
```

```
{      cout<<"a="<<a<<",b="<<b<<",c="<<c<<",that is not a trianle!\n"; }
```

```
}
```

观察一下，

抛出（throw）

检查（try）

捕捉（catch）

分别出现在什么位置，思考一下是否合理

一、异常

```
try
{
    // 保护代码
}
catch( ExceptionName e1 )
{
    // catch 块
}
catch( ExceptionName e2 )
{
    // catch 块
}
catch( ExceptionName eN )
{
    // catch 块
}
```



```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
void Demo1()
```

```
{ try { throw "hello"; }
```

```
catch(char) { cout << "catch(char)" << endl;}
```

```
catch(int) { cout << "catch(int)" << endl; }
```

```
catch(double) { cout << "catch(double)" << endl;}
```

```
catch(...) // 表示捕获任意类型的异常
```

```
{ cout << "catch(...)" << endl; }
```

捕获任意异常/捕获所有异常，或叫匿名捕获异常

```
}
```

```
void Demo2()
```

```
{ //throw string("D.T.Software");
```

```
throw "hello"; }
```

```
void main()
```

```
{ Demo1();
```

```
try { Demo2(); }
```

```
catch(char* ) { cout << "catch(char *)" << endl; }
```

```
catch(string) { cout << "catch(string)" << endl; }
```

```
}
```



```
#include <iostream>
using namespace std;
double division(double a, double b)
{ if( b == 0 )
  { throw 'a'; }
  return (a/b);
}
void main ()
{ double x,y;
  double z;
  cin>>x>>y;
  try
  { z = division(x, y);
    cout << z << endl;
  }catch (char)
  { cerr <<"Division by zero condition!"<< endl; }
}
```

输入

1

2

输入

1

0



一、异常

```
#include <iostream.h>
void f1()
{ void f2();
  try
  {f2();}
  catch(char)
  { cout<<"ERROR1!\n";    }
  cout<<"end1"<<endl;
}
void f2()
{ void f3();
  try
  {f3();}
  catch(int)
  { cout<<"ERROR2!\n";    }
  cout<<"end2"<<endl;
}
```

```
void f3()
{ double a=0;
  try {throw a;}
  catch(float)
  { cout<<"ERROR3!\n";
    cout<<"end3"<<endl;
  }
}
void main()
{ try{
      f1();  }
  catch(double)
  { cout<<"ERROR0!\n";    }
  cout<<"end0"<<endl;
}
```

尝试修改

double a=0;为

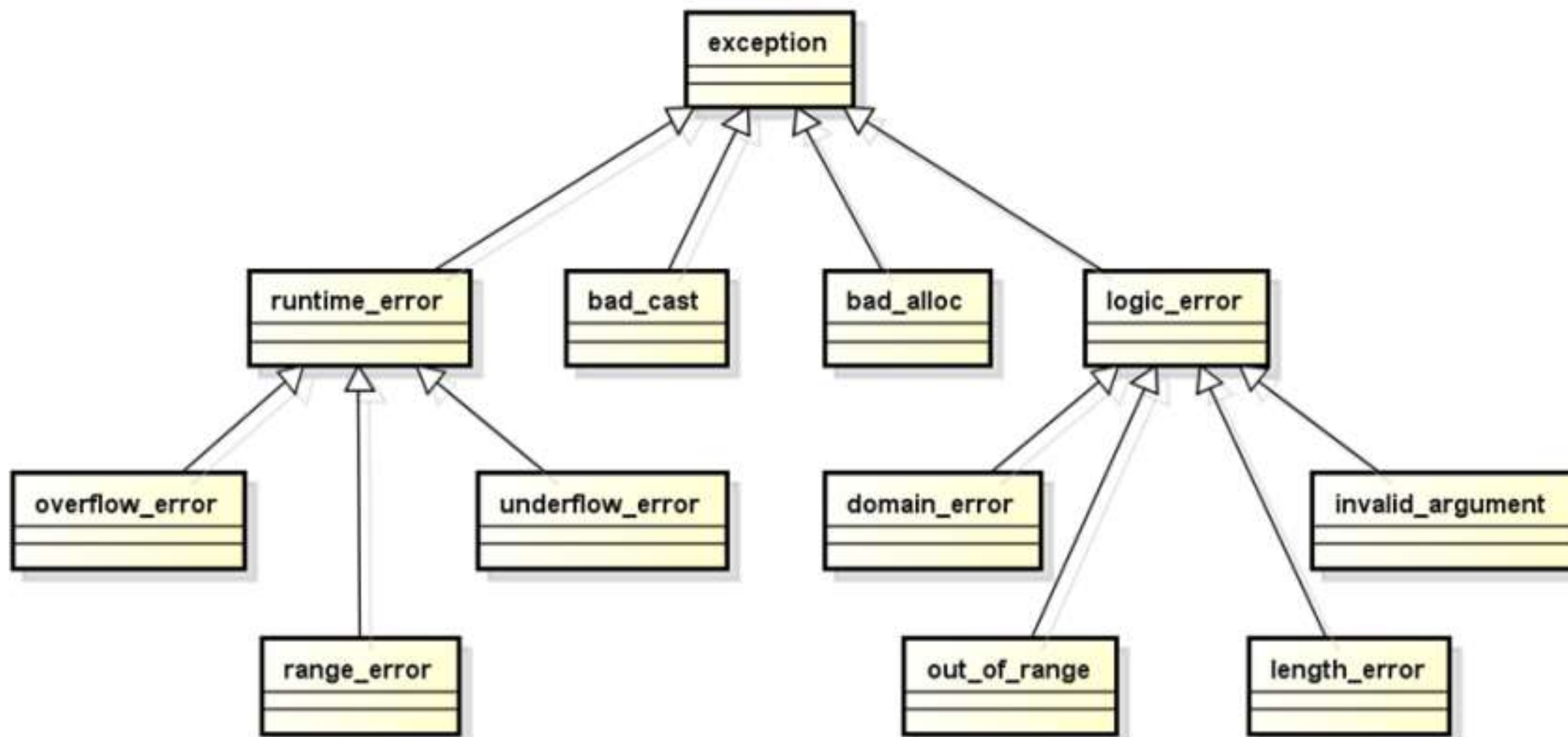
float a=0;

char a='a';



一、异常

C++ 提供一系列标准异常，定义在 `<exception>` 中，在程序中使用标准异常



一、异常

异常	描述
exception	所有标准 C++ 异常的父类
bad_alloc	new 抛出
bad_cast	通过 dynamic_cast 抛出
bad_exception	在处理 C++ 程序中无法预期的异常时非常有用
logic_error	可通过读取代码来检测到的异常
invalid_argument	当使用了无效的参数时，会抛出该异常
length_error	当创建了太长的 string 时，会抛出该异常
out_of_range	该异常可以通过方法抛出，例如 vector
runtime_error	理论上不可以通过读取代码来检测到的异常
overflow_error	当发生数学上溢时，会抛出该异常
range_error	当尝试存储超出范围的值时，会抛出该异常
underflow_error	当发生数学下溢时，会抛出该异常



一、异常

```
#include <iostream>
#include <exception>
using namespace std;
class MyException : public exception
{public:
    char * what ()
    {   return "C++ Exception"; }
};
void main()
{ try {   throw MyException(); }
  catch(MyException& e)
  {   cout << "MyException caught" << endl;
      cout << e.what() << endl; }
  catch(exception& e) //其他的错误
  {   cout << "other exception caught" << endl; }
}
```

通过继承和重载

exception 类定义新异常

what() 函数返回能识别异常的字符串，可粗略地知道是什么异常



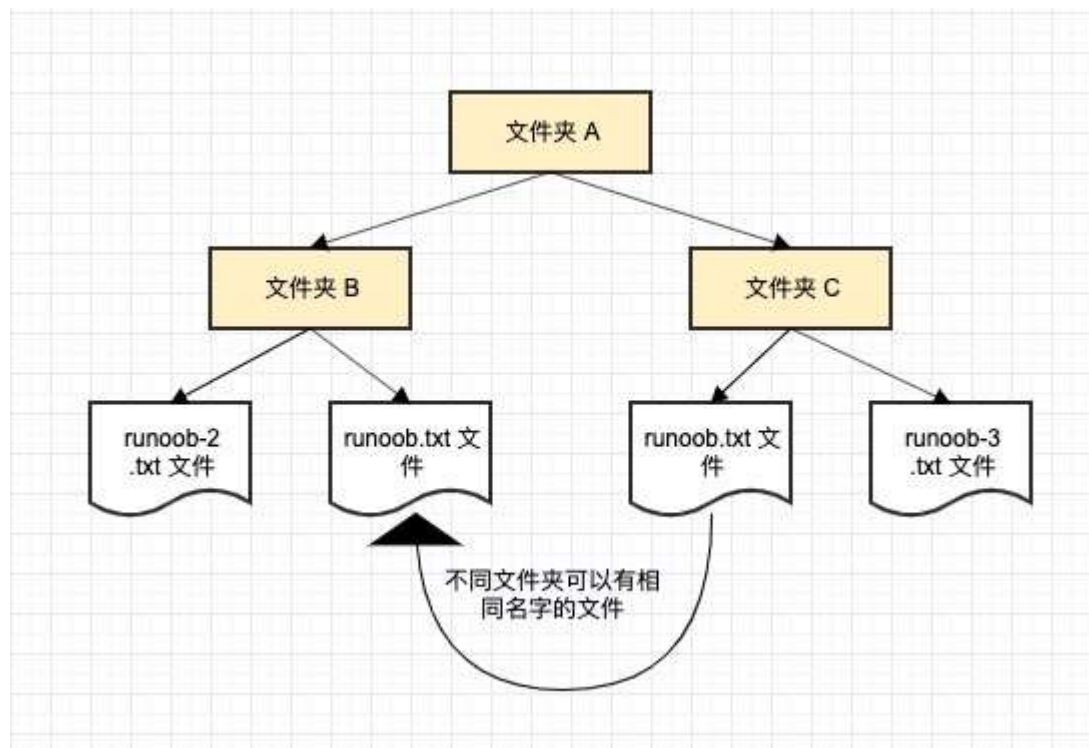
一、异常

```
#include <iostream>
#include <string>
#include <stdexcept>
using namespace std;
void main()
{
    string str("foo");
    try // access element, may throw out_of_range
    { str.at(10); }
    catch (const out_of_range& e)
    // what() is inherited from exception and contains an explanatory message
    { cout << e.what(); }
}
```



二、名字空间

命名空间就是定义了一个范围



二、名字空间

命名空间的定义

```
namespace namespace_name { // 代码声明 }
```

调用带有命名空间的函数或变量，需要在前面加上命名空间的名称

```
name::code; // code 可以是变量或函数
```



二、名字空间

```
#include <iostream>
using namespace std;
// 第一个命名空间
namespace first_space
{ void func()
  { cout << "Inside first_space" << endl; }
}
// 第二个命名空间
namespace second_space
{ void func()
  { cout << "Inside second_space" << endl; }
}
void main ()
{ // 调用第一个命名空间中的函数
  first_space::func();
  // 调用第二个命名空间中的函数
  second_space::func();
}
```



二、名字空间

用 `using namespace` 指令，在使用命名空间时可不用在前面加上命名空间的名称。指示编译器，后续的代码将使用指定的命名空间中的名称

```
#include <iostream>
using namespace std;
// 第一个命名空间
namespace first_space
{ void func()
  { cout << "Inside first_space" << endl; } }
// 第二个命名空间
namespace second_space
{ void func()
  { cout << "Inside second_space" << endl; } }
using namespace second_space;
void main ()
{func();}
```



二、名字空间

using 指令可用来指定命名空间中特定项目

例，如只使用 **std** 命名空间中的 **cout**

```
#include <iostream>
using std::cout;
void main ()
{
    cout << "std::endl is used with std!" << endl;
    // cout << "std::endl is used with std!" << std::endl;
}
```



谢 谢

